Practice Session on Stack and Introduction to Queues

Stack Implementation in Java

What is a Stack?

A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. The last element added to the stack is the first one to be removed.

Basic Stack Operations

- **Push**: Add an element to the top of the stack
- **Pop**: Remove and return the top element from the stack
- **Peek/Top**: Return the top element without removing it
- **isEmpty**: Check if the stack is empty
- **Size**: Get the number of elements in the stack

Stack Implementation Using Arrays

java

```java
class Stack {
    private int[] stack;
    private int top;
    private int capacity;

    // Constructor to initialize stack
    public Stack(int size) {
        stack = new int[size];
        capacity = size;
        top = -1;
    }

    // Push operation
    public void push(int item) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow");
            return;
        }
        stack[++top] = item;
        System.out.println(item + " pushed to stack");
    }

    // Pop operation
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return stack[top--];
    }

    // Peek operation
    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack is empty");
            return -1;
        }
        return stack[top];
    }

    // Check if stack is empty
    public boolean isEmpty() {
        return top == -1;
```

```java
    }

    // Get stack size
    public int size() {
        return top + 1;
    }
}
```

Using Java's Built-in Stack Class

```java
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // Push elements
        stack.push(10);
        stack.push(20);
        stack.push(30);

        // Display stack
        System.out.println("Stack: " + stack);

        // Pop element
        int poppedElement = stack.pop();
        System.out.println("Popped: " + poppedElement);

        // Peek top element
        int topElement = stack.peek();
        System.out.println("Top element: " + topElement);

        // Check if empty
        System.out.println("Is empty: " + stack.isEmpty());

        // Get size
        System.out.println("Size: " + stack.size());
    }
}
```

Introduction to Queues

What is a Queue?

A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. The first element added to the queue is the first one to be removed.

Basic Queue Operations

- **Enqueue**: Add an element to the rear of the queue
- **Dequeue**: Remove and return the front element from the queue
- **Front**: Return the front element without removing it
- **isEmpty**: Check if the queue is empty
- **Size**: Get the number of elements in the queue

Queue Implementation Using Arrays

java

```java
class Queue {
    private int[] queue;
    private int front;
    private int rear;
    private int capacity;
    private int size;

    // Constructor to initialize queue
    public Queue(int capacity) {
        this.capacity = capacity;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    // Enqueue operation
    public void enqueue(int item) {
        if (size == capacity) {
            System.out.println("Queue is full");
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = item;
        size++;
        System.out.println(item + " enqueued to queue");
    }

    // Dequeue operation
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        int item = queue[front];
        front = (front + 1) % capacity;
        size--;
        return item;
    }

    // Front operation
    public int front() {
        if (isEmpty()) {
```

```java
            System.out.println("Queue is empty");
            return -1;
        }
        return queue[front];
    }

    // Check if queue is empty
    public boolean isEmpty() {
        return size == 0;
    }

    // Get queue size
    public int size() {
        return size;
    }
}
```

Using Java's Built-in Queue Interface

```java
import java.util.Queue;
import java.util.LinkedList;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();

        // Enqueue elements
        queue.offer(10);
        queue.offer(20);
        queue.offer(30);

        // Display queue
        System.out.println("Queue: " + queue);

        // Dequeue element
        int dequeuedElement = queue.poll();
        System.out.println("Dequeued: " + dequeuedElement);

        // Peek front element
        int frontElement = queue.peek();
        System.out.println("Front element: " + frontElement);

        // Check if empty
        System.out.println("Is empty: " + queue.isEmpty());

        // Get size
        System.out.println("Size: " + queue.size());
    }
}
```

Key Differences Between Stack and Queue

Aspect	Stack	Queue
Principle	LIFO (Last-In-First-Out)	FIFO (First-In-First-Out)
Insertion	Push at top	Enqueue at rear
Deletion	Pop from top	Dequeue from front
Access	Only top element	Only front element

Quiz Questions

1. What principle does a Stack follow? a) FIFO (First-In-First-Out) b) LIFO (Last-In-First-Out) c) Random Access d) Sequential Access

Answer: b) LIFO (Last-In-First-Out)

2. Which method is used to add an element to a Stack in Java's built-in Stack class? a) add() b) insert() c) push() d) enqueue()

Answer: c) push()

3. What happens when you try to pop from an empty stack? a) Returns null b) Returns 0 c) Throws an exception or shows underflow d) Nothing happens

Answer: c) Throws an exception or shows underflow

4. In a Queue, where are new elements added? a) At the front b) At the rear c) In the middle d) Anywhere

Answer: b) At the rear

5. Which of the following correctly represents the order of elements removed from a queue containing [1, 2, 3, 4] (where 1 was added first)? a) 4, 3, 2, 1 b) 1, 2, 3, 4 c) 2, 1, 4, 3 d) Random order

Answer: b) 1, 2, 3, 4

Practice Coding Questions

1. Write a Java program to implement a stack that can store String values. Include methods for push, pop, peek, isEmpty, and display all elements.

2. Create a method called `reverseString(String str)` that uses a stack to reverse the given string and returns the reversed string.

3. Implement a queue using two stacks. Your implementation should support enqueue and dequeue operations.

4. Write a Java program to check if parentheses in a given string are balanced using a stack. For example, "(())" should return true, while "(()" should return false.

5. Create a circular queue implementation with methods enqueue, dequeue, front, isEmpty, and isFull. The queue should have a fixed capacity and handle wraparound correctly.